



Détection des Messages Indésirables

Classification automatique des spams

Présenté par :

ANDRIAMIHAJAMANANA Anjara

N° 144/LA/25-26

Année Universitaire : 2025 – 2026

Table de matières

I- Analyse et préparation de données	3
1- Chargement et Exploration de données	3
2- Nettoyage de donnée	4
3- Feature Engineering	4
II- Conception et Entraînement des Modèles	5
1- Division du jeu de donnée	5
2- Modélisation et Entraînement	5
III- Évaluation et Analyse des Résultats	5
1- Resultats des performances	5
2- Comparaison des Modèles	7
IV- Conclusion	8

Introduction

Dans le monde d'aujourd'hui, la communication numérique est devenue le pilier central des échanges personnels, académiques et professionnels. Cependant, cette connectivité omniprésente s'accompagne d'une prolifération massive de messages non sollicités, communément appelés "spams". Qu'il s'agisse de publicités agressives, de tentatives d'hameçonnage (phishing) ou de la diffusion de logiciels malveillants, les spams encombrant les infrastructures réseau, saturant les espaces de stockage et représentent une menace constante pour la sécurité des données des utilisateurs.

Face à des volumes de données de plus en plus gigantesques, les méthodes de filtrage traditionnelles basées sur de simples listes de mots-clés sont devenues obsolètes. Les expéditeurs de spams rivalisent d'ingéniosité pour contourner les règles statiques. C'est dans ce contexte que l'apprentissage automatique s'impose comme une solution incontournable.

L'objectif de ce projet est de concevoir et d'implémenter un système de classification automatique capable de distinguer avec précision les messages légitimes (*ham*) des messages indésirables (*spam*). Pour y parvenir, notre approche repose sur une étude comparative de trois algorithmes éprouvés en traitement automatique du langage naturel (NLP) : la Régression Logistique, les Support Vector Machines (SVM), et le modèle Multinomial Naive Bayes. En analysant les structures textuelles et sémantiques des données à travers ces trois prismes, ce travail vise à identifier le modèle offrant le meilleur compromis entre la vitesse d'exécution et la précision du filtrage.

I- Analyse et préparation de données

1- Chargement et Exploration de données

la source de la donnée que nous allons utiliser provient de *kaggle.com*. Cette donnée est une collection de messages SMS étiquetté rassemblé pour la recherche des SMS spam. Chaque message a une étiquette soit **ham**(legitime) ou **spam**(indésirable).

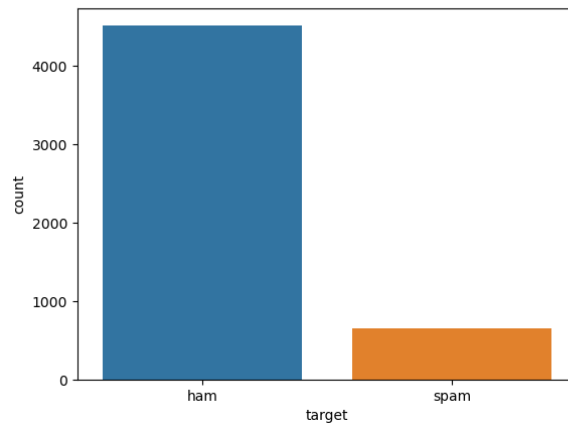


figure 1.1

Dans notre jeu de donnée , nous avons au total 5169 messages dont 4516 sont des messages legitimes et 653 d'eux sont des messages indésirables.

Notre but pour cette modèle est de classifier les messages comme **ham** ou **spam** ; il est important de savoir ce qui fait de ces messages classifié ham ou spam .

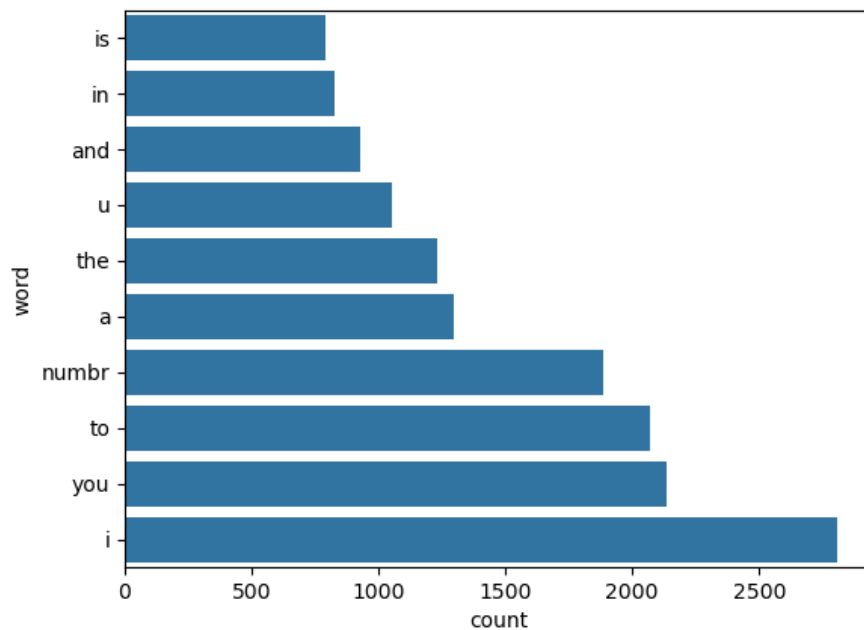


figure 1.2

La *figure 1.2* affiche les mots les plus fréquents dans le jeu de donnée. On remarque tout de suite que les mots les plus fréquents sont des mots comme 'i' , 'you' , 'to' , 'and' , ... Or , ces mots n'aident même pas dans la classification des messages. Ainsi , ces mots ne sont alors que des bruits dans notre jeu de donnée ; ils sont considérés comme des **mots vides**.

2- Nettoyage de donnée

On doit alors supprimer de notre jeu de donnée ces **mots vides**.

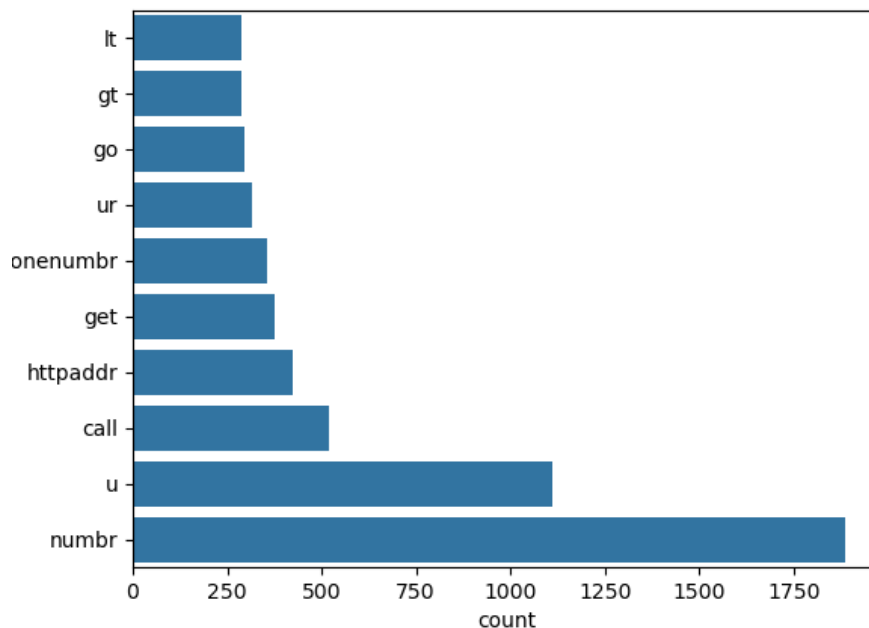


figure 1.3

Même si le jeu de donnée est moins bruyants , on peut encore soigner notre jeu de donnée grâce à une technique appelée **lemmatisation**. La lemmatisation consiste à réduire les mots dans les messages à son mot correspondant dans le dictionnaire.

Après la lemmatisation , les messages sont bien soignés et procèdent à leur transformation numérique, cette étape s'appelle le **feature engineering**.

3- Feature Engineering

Pour transformer les messages dans le jeu de donnée , on utilise la méthode de **TF-IDF**. TF-IDF est l'abréviation de *Term Frequency* et *Inverse Document Frequency*. Une méthode qui numérise les messages tout en considérant leur signification sémantique.

Pour être plus technique , on combine **word-level TF-IDF avec ngram de (1,2)** et **char-level TF-IDF avec ngram de (3,5)**

- **word-level TF-IDF avec ngram de (1,2)** : Une méthode en prenant tous les mots et les mots deux à deux proche avant d'appliquer la TF-IDF
- **char-level TF-IDF avec ngram de (3,5)** : Une méthode en prenant tous les lettres de longueur 3 à 5 pour chaque mots avant d'appliquer la TF-IDF

Avec cette méthode , on peut facilement capturer les messages indésirables avancés comme **mon3y** ou **w!n f r 3 e mon3y** ou **click here** !

II- Conception et Entraînement des Modèles

1- Division du jeu de donnée

Après le **feature engineering**, notre jeu de donnée est maintenant numérique et prêt à être entraîné dans des modèles. Mais avant cela, séparons notre jeu de donnée en **donnée d'entraînement** et **donnée de test**. La raison pour la séparation de donnée est l'évaluation du modèle après son entraînement. Si on ne sépare pas le jeu de donnée, on aurait aucun moyen pour savoir si notre modèle est mauvais ou bien.

2- Modélisation et Entraînement

On est maintenant sur la création et l'entraînement du modèle. Les modèles qui vont être utilisés pour notre jeu de donnée sont :

- SVM
- Multinomial Naive Bayes
- Logistic Regression

III- Évaluation et Analyse des Résultats

1- Résultats des performances

Après l'entraînement des trois modèles, nous avons leurs résultats métriques.

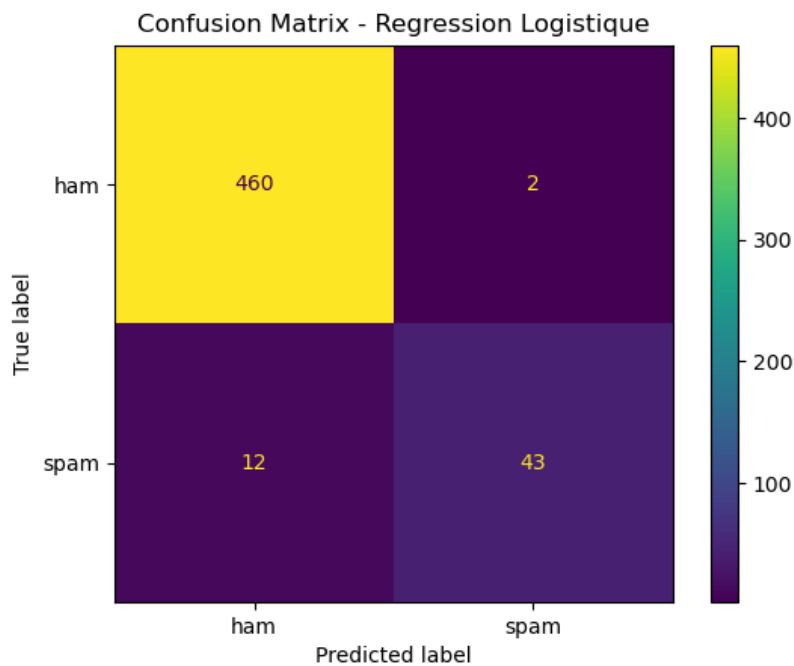


figure 1.4

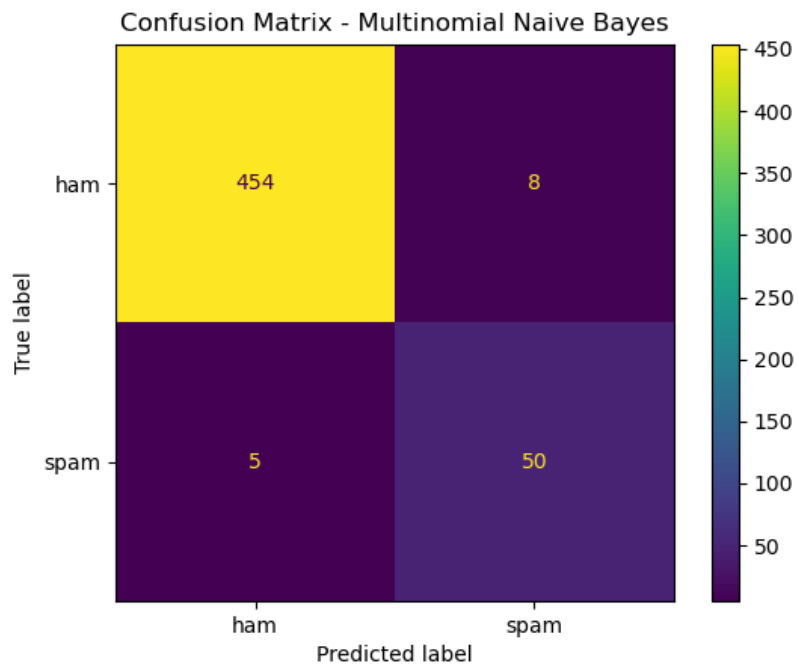


figure 1.5

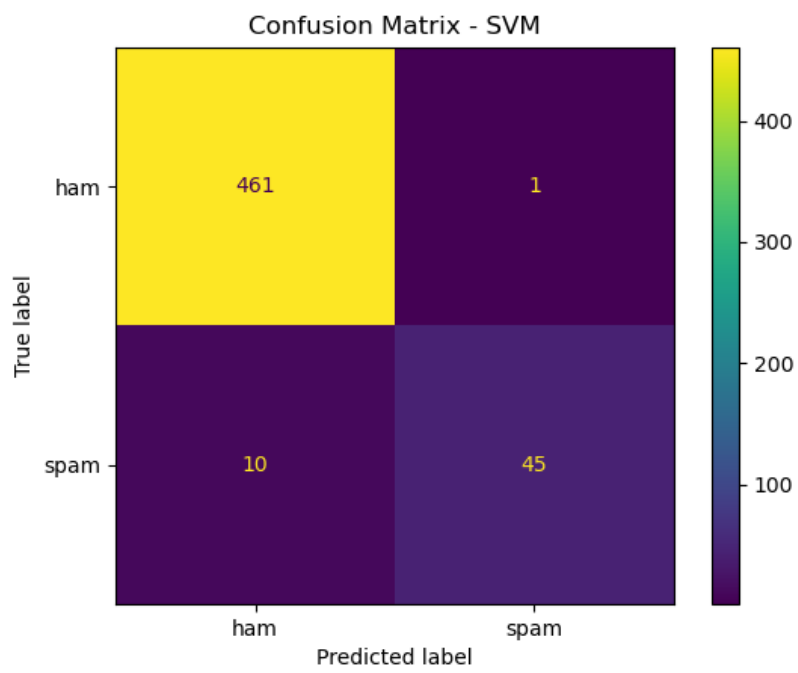


figure 1.6

La **figure 1.7** affiche les métriques des trois modèles en pourcentage.

	model_name	accuracy	precision	recall	f1
0	SVM	97.87	97.83	81.82	89.11
1	Multinomial Naive Bayes	97.49	86.21	90.91	88.50
2	Regression Logistique	97.29	95.56	78.18	86.00

figure 1.7

2- Comparaison des Modèles

À partir de ces performances, créons un graphe plus intuitive avec leur *accuracy* , *precision* , *recall* et *f1-score*

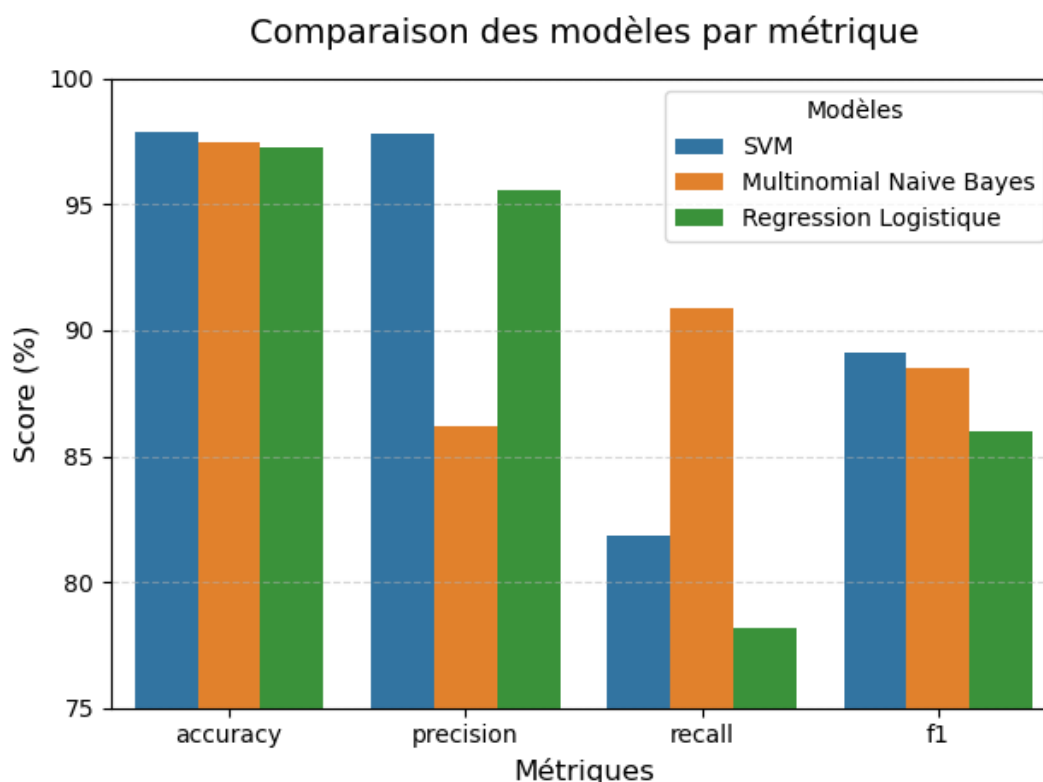


figure 1.6

On choisit le modèle qui convient mieux par rapport au contexte.

- Si on veut minimiser le nombre de spam , on utilise **Multinomial Naive Bayes** car il est le modèle avec le plus de **recall** (90,91 %)
- Si les messages non-spam sont importants mais les messages spam sont toujours à minimiser, on utilise **SVM** car il est le modèle avec le plus de **precision**(97,83 %) et **accuracy**(97,87 %)
- Si on veut privilégier la rapidité de notre modèle, on utilise **Regression Logistique**

IV- Conclusion

En conclusion, ce projet nous a permis de concevoir et de comparer rigoureusement trois approches d'apprentissage automatique pour la classification des messages indésirables.

L'implémentation de la chaîne de traitement textuel, combinée à la méthode de vectorisation TF-IDF (Term Frequency-Inverse Document Frequency), s'est avérée essentielle pour transformer efficacement les données textuelles brutes en caractéristiques numériques exploitables par nos algorithmes.

L'étude comparative a mis en lumière les forces de chaque modèle selon les métriques d'évaluation retenues :

- La **Régression Logistique** s'est imposée comme une excellente solution de référence , offrant une très bonne exactitude globale et une vitesse de calcul remarquable.
- Le modèle **Multinomial Naive Bayes** a confirmé sa réputation historique en classification textuelle en affichant un excellent compromis global et un score de *Rappel* élevé, idéal pour intercepter un maximum de spams.
- Les **SVM** se sont distinguées par leur régularité et leur haute *Précision*, une qualité critique pour un filtre de messagerie afin de minimiser le taux de faux positifs et d'éviter la perte de messages légitimes importants.

Bien que les performances obtenues soient hautement satisfaisantes pour un déploiement standard, plusieurs perspectives d'amélioration peuvent être envisagées. À court terme, l'ajustement plus fin des hyperparamètres avec *GridSearch*. À plus long terme, l'exploration de modèles de Deep Learning ou de techniques basées sur des transformeurs tels que BERT permettrait de capturer le contexte sémantique profond des messages et de mieux contrer les stratégies d'évasion des expéditeurs de spams.